

PAULT DOCUMENTATION

2026 02 21 Versioning V2

Source: docs/plans/2026-02-21-versioning-v2.md

Theme: Clean Technical

Generated: February 26, 2026

Versioning v2 Implementation Plan

> **For Claude:** REQUIRED SUB-SKILL: Use superpowers:executing-plans to implement this plan task-by-task.

Goal: Overhaul Prompt Versioning with line+character diffs, metadata snapshots, version management, and a restore-preview workflow.

Architecture: New DiffEngine (pure logic, no UI) provides two-pass line+character diffing using Swift's built-in CollectionDifference. PromptVersion gains isFavorite: Bool and snapshotData: Data? (JSON-encoded VersionSnapshot for tags/variables). UI overhauled for inline/side-by-side diff toggle, version management (delete, search), and restore preview with partial field selection.

Tech Stack: SwiftUI, SwiftData, Swift Testing (@Test), CollectionDifference API

Task 1: VersionSnapshot Codable Struct + PromptVersion Model Changes

Files:

- Modify: Pault/PromptVersion.swift
- Test: PaultTests/Models/PromptVersionTests.swift

Step 1: Write failing tests for VersionSnapshot encoding/decoding and new model fields

Add these tests to PaultTests/Models/PromptVersionTests.swift:

```
@Test func versionSnapshot_roundTrips() throws {
    let snapshot = VersionSnapshot(
        tags: [.init(name: "coding", color: "blue"), .init(name: "work", color: "green")],
        variables: [.init(name: "topic", defaultValue: "AI", occurrenceIndex: 0)]
    )
    let data = try JSONEncoder().encode(snapshot)
    let decoded = try JSONDecoder().decode(VersionSnapshot.self, from: data)

    #expect(decoded.tags.count == 2)
    #expect(decoded.tags[0].name == "coding")
    #expect(decoded.tags[0].color == "blue")
    #expect(decoded.variables.count == 1)
    #expect(decoded.variables[0].name == "topic")
    #expect(decoded.variables[0].defaultValue == "AI")
    #expect(decoded.variables[0].occurrenceIndex == 0)
}

@Test func versionSnapshot_decodesWithMissingOptionals() throws {
    // Simulates a version saved before new fields were added
    let json = Data("#{"tags":[],"variables":[]}#.utf8)
    let decoded = try JSONDecoder().decode(VersionSnapshot.self, from: json)
    #expect(decoded.tags.isEmpty)
    #expect(decoded.variables.isEmpty)
}

@Test func promptVersion_newFieldsHaveDefaults() throws {
    let container = try makeContainer()
    let ctx = ModelContext(container)
    let version = PromptVersion(title: "T", content: "C")
    ctx.insert(version)
```

```

try ctx.save()

#expect(version.isFavorite == false)
#expect(version.snapshotData == nil)
}

@Test func promptVersion_snapshotComputedProperty() throws {
    let version = PromptVersion(title: "T", content: "C")
    #expect(version.snapshot == nil) // no data yet

    let snap = VersionSnapshot(
        tags: [.init(name: "test", color: "red")],
        variables: []
    )
    version.snapshot = snap

    #expect(version.snapshotData != nil)
    #expect(version.snapshot?.tags.count == 1)
    #expect(version.snapshot?.tags[0].name == "test")
}

```

Step 2: Run tests to verify they fail

Run: `xcodebuild test -scheme Pault -destination 'platform=macOS' -only-testing:PaultTests/PromptVersionTests 2>&1 | tail -20`

Expected: Compilation errors — `VersionSnapshot` and new fields don't exist yet.

Step 3: Implement `VersionSnapshot` and model changes

Replace `Pault/PromptVersion.swift` with:

```

import Foundation
import SwiftData

/// Codable snapshot of prompt metadata captured at version-save time.
struct VersionSnapshot: Codable, Equatable {
    var tags: [TagSnapshot]
    var variables: [VariableSnapshot]

    struct TagSnapshot: Codable, Equatable {
        var name: String
        var color: String
    }

    struct VariableSnapshot: Codable, Equatable {
        var name: String
        var defaultValue: String
        var occurrenceIndex: Int
    }
}

@Model
final class PromptVersion {
    var id: UUID
    @Relationship(deleteRule: .nullify) var prompt: Prompt?
    var title: String
    var content: String
    var savedAt: Date
}

```

```

var changeNote: String?
var isFavorite: Bool
var snapshotData: Data?

/// Convenience computed property to encode/decode the VersionSnapshot.
var snapshot: VersionSnapshot? {
  get {
    guard let data = snapshotData else { return nil }
    return try? JSONDecoder().decode(VersionSnapshot.self, from: data)
  }
  set {
    snapshotData = newValue.flatMap { try? JSONEncoder().encode($0) }
  }
}

init(
  id: UUID = UUID(),
  prompt: Prompt? = nil,
  title: String,
  content: String,
  savedAt: Date = Date(),
  changeNote: String? = nil,
  isFavorite: Bool = false,
  snapshotData: Data? = nil
) {
  self.id = id
  self.prompt = prompt
  self.title = title
  self.content = content
  self.savedAt = savedAt
  self.changeNote = changeNote
  self.isFavorite = isFavorite
  self.snapshotData = snapshotData
}

```

Step 4: Run tests to verify they pass

Run: `xcodebuild test -scheme Pault -destination 'platform=macOS' -only-testing:PaultTests/PromptVersionTests 2>&1 | tail -20`

Expected: All tests PASS (existing + new).

Step 5: Commit

```

git add Pault/PromptVersion.swift PaultTests/Models/PromptVersionTests.swift
git commit -m "feat(versioning): add VersionSnapshot Codable struct and
isFavorite/snapshotData fields"

```

Task 2: Enriched saveSnapshot + Dedup Guard

Files:

- Modify: `Pault/PromptService.swift:199-227`
- Test: `PaultTests/PromptVersionSnapshotTests.swift`

Step 1: Write failing tests for enriched snapshots and dedup guard

Add to `PaulTests/PromptVersionSnapshotTests.swift`:

```
// MARK: - saveSnapshot_capturesMetadata

@Test func saveSnapshot_capturesTagsAndVariables() throws {
    let context = try makeContext()
    let service = PromptService(modelContext: context)

    let prompt = service.createPrompt(title: "Test", content: "Hello {{name}}")
    TemplateEngine.syncVariables(for: prompt, in: context)
    prompt.templateVariables.first?.defaultValue = "World"

    let tag = service.createTag(name: "demo", color: "purple")
    service.addTag(tag, to: prompt)

    service.saveSnapshot(for: prompt)

    let descriptor = FetchDescriptor<PromptVersion>()
    let versions = try context.fetch(descriptor)
    let version = try #require(versions.first)

    #expect(version.isFavorite == false)
    let snap = try #require(version.snapshot)
    #expect(snap.tags.count == 1)
    #expect(snap.tags[0].name == "demo")
    #expect(snap.tags[0].color == "purple")
    #expect(snap.variables.count == 1)
    #expect(snap.variables[0].name == "name")
    #expect(snap.variables[0].defaultValue == "World")
}

@Test func saveSnapshot_capturesFavoriteStatus() throws {
    let context = try makeContext()
    let service = PromptService(modelContext: context)

    let prompt = service.createPrompt(title: "Fav", content: "Body")
    prompt.isFavorite = true

    service.saveSnapshot(for: prompt)

    let descriptor = FetchDescriptor<PromptVersion>()
    let version = try #require(try context.fetch(descriptor).first)

    #expect(version.isFavorite == true)
}

// MARK: - Dedup Guard

@Test func saveSnapshot_skipsWhenNothingChanged() throws {
    let context = try makeContext()
    let service = PromptService(modelContext: context)

    let prompt = service.createPrompt(title: "Same", content: "Same")
    service.saveSnapshot(for: prompt)
    service.saveSnapshot(for: prompt) // duplicate – should be skipped

    let descriptor = FetchDescriptor<PromptVersion>()
    let versions = try context.fetch(descriptor)
    #expect(versions.count == 1)
```

```

}

@Test func saveSnapshot_createsWhenContentChanges() throws {
    let context = try makeContext()
    let service = PromptService(modelContext: context)

    let prompt = service.createPrompt(title: "Title", content: "v1")
    service.saveSnapshot(for: prompt)

    prompt.content = "v2"
    service.saveSnapshot(for: prompt)

    let descriptor = FetchDescriptor<PromptVersion>()
    let versions = try context.fetch(descriptor)
    #expect(versions.count == 2)
}

@Test func saveSnapshot_createsWhenFavoriteChanges() throws {
    let context = try makeContext()
    let service = PromptService(modelContext: context)

    let prompt = service.createPrompt(title: "T", content: "C")
    service.saveSnapshot(for: prompt)

    prompt.isFavorite = true
    service.saveSnapshot(for: prompt)

    let descriptor = FetchDescriptor<PromptVersion>()
    let versions = try context.fetch(descriptor)
    #expect(versions.count == 2)
}

```

Step 2: Run tests to verify they fail

Run: `xcodebuild test -scheme Pault -destination 'platform=macOS' -only-testing:PaultTests/PromptVersionSnapshotTests 2>&1 | tail -20`

Expected: FAIL — metadata not captured, dedup guard not implemented.

Step 3: Update saveSnapshot in PromptService

Replace the saveSnapshot method at `Pault/PromptService.swift:199-227` with:

```

func saveSnapshot(for prompt: Prompt, changeNote: String? = nil, limit: Int = 50) {
    // Dedup guard: skip if nothing changed vs latest version
    let promptID = prompt.id
    let descriptor = FetchDescriptor<PromptVersion>(
        sortBy: [SortDescriptor(\.savedAt, order: .reverse)]
    )
    if let allVersions = try? modelContext.fetch(descriptor),
        let latest = allVersions.first(where: { $0.prompt?.id == promptID }) {
        let currentTagNames = prompt.tags.map(\.name).sorted()
        let latestTagNames = latest.snapshot?.tags.map(\.name).sorted() ?? []
        let currentVarPairs = prompt.templateVariables
            .sorted(by: { $0.sortOrder < $1.sortOrder })
            .map { "\( $0.name )=\( $0.defaultValue )" }
        let latestVarPairs = latest.snapshot?.variables
            .sorted(by: { $0.occurrenceIndex < $1.occurrenceIndex })
            .map { "\( $0.name )=\( $0.defaultValue )" } ?? []
    }
}

```

```

let unchanged = prompt.content == latest.content
&& prompt.title == latest.title
&& prompt.isFavorite == latest.isFavorite
&& currentTagNames == latestTagNames
&& currentVarPairs == latestVarPairs

if unchanged && changeNote == nil { return }
}

// Build snapshot metadata
let tagSnapshots = prompt.tags.map {
VersionSnapshot.TagSnapshot(name: $0.name, color: $0.color)
}
let varSnapshots = prompt.templateVariables
.sorted(by: { $0.sortOrder < $1.sortOrder })
.map {
VersionSnapshot.VariableSnapshot(
name: $0.name,
defaultValue: $0.defaultValue,
occurrenceIndex: $0.occurrenceIndex
)
}
let snapshot = VersionSnapshot(tags: tagSnapshots, variables: varSnapshots)

let version = PromptVersion(
prompt: prompt,
title: prompt.title,
content: prompt.content,
changeNote: changeNote,
isFavorite: prompt.isFavorite,
snapshotData: try? JSONEncoder().encode(snapshot)
)
modelContext.insert(version)

// Prune: keep only the most recent `limit` versions for this prompt.
let allDescriptor = FetchDescriptor<PromptVersion>(
sortBy: [SortDescriptor(\.savedAt, order: .forward)]
)
guard let allVersions = try? modelContext.fetch(allDescriptor) else { return }
let promptVersions = allVersions.filter { $0.prompt?.id == promptID }

if promptVersions.count > limit {
let toDelete = promptVersions.prefix(promptVersions.count - limit)
for v in toDelete {
modelContext.delete(v)
}
}
save("saveSnapshot")
}

```

Step 4: Run ALL snapshot tests

Run: `xcodebuild test -scheme Pault -destination 'platform=macOS' -only-testing:PaultTests/PromptVersionSnapshotTests 2>&1 | tail -30`

Expected: All tests PASS.

Step 5: Commit

```
git add Pault/PromptService.swift PaultTests/PromptVersionSnapshotTests.swift
git commit -m "feat(versioning): enrich saveSnapshot with metadata capture and dedup
guard"
```

Task 3: DiffEngine (Pure Logic)

Files:

- Create: Pault/DiffEngine.swift
- Create: PaultTests/DiffEngineTests.swift

Step 1: Write failing tests

Create PaultTests/DiffEngineTests.swift:

```
import Testing
@testable import Pault

struct DiffEngineTests {

    // MARK: - Identical texts

    @Test func diff_identicalTexts_allUnchanged() {
        let result = DiffEngine.diff(old: "hello\nworld", new: "hello\nworld")
        #expect(result.allSatisfy { $0.kind == .unchanged })
        #expect(result.count == 2)
    }

    // MARK: - Empty inputs

    @Test func diff_emptyOldText_allAdded() {
        let result = DiffEngine.diff(old: "", new: "line one\nline two")
        #expect(result.filter { $0.kind == .added }.count == 2)
        #expect(result.filter { $0.kind == .removed }.count == 0)
    }

    @Test func diff_emptyNewText_allRemoved() {
        let result = DiffEngine.diff(old: "line one\nline two", new: "")
        #expect(result.filter { $0.kind == .removed }.count == 2)
        #expect(result.filter { $0.kind == .added }.count == 0)
    }

    @Test func diff_bothEmpty_noResults() {
        let result = DiffEngine.diff(old: "", new: "")
        #expect(result.isEmpty)
    }

    // MARK: - Single line change

    @Test func diff_singleLineChanged_hasCharacterDiffs() {
        let result = DiffEngine.diff(old: "hello world", new: "hello earth")
        // Should produce a removed + added pair with character-level diffs
        let removed = result.filter { $0.kind == .removed }
        let added = result.filter { $0.kind == .added }
        #expect(removed.count == 1)
        #expect(added.count == 1)
    }
}
```



```

#expect(removed[0].characterDiffs != nil)
#expect(added[0].characterDiffs != nil)
}

// MARK: - Multi-line changes

@Test func diff_addedLine_detectedCorrectly() {
    let old = "line one\nline three"
    let new = "line one\nline two\nline three"
    let result = DiffEngine.diff(old: old, new: new)
    let added = result.filter { $0.kind == .added }
    #expect(added.count == 1)
    #expect(added[0].text == "line two")
}

@Test func diff_removedLine_detectedCorrectly() {
    let old = "line one\nline two\nline three"
    let new = "line one\nline three"
    let result = DiffEngine.diff(old: old, new: new)
    let removed = result.filter { $0.kind == .removed }
    #expect(removed.count == 1)
    #expect(removed[0].text == "line two")
}

// MARK: - Character-level refinement

@Test func characterDiff_identifiesChangedWord() {
    let charDiffs = DiffEngine.characterDiff(old: "the quick brown fox", new: "the slow brown fox")
    let changed = charDiffs.filter { $0.kind != .unchanged }
    #expect(!changed.isEmpty)
}
}

```

Step 2: Run tests to verify they fail

Run: `xcodebuild test -scheme Pault -destination 'platform=macOS' -only-testing:PaultTests/DiffEngineTests 2>&1 | tail -20`

Expected: Compilation error — DiffEngine does not exist.

Step 3: Implement DiffEngine

Create `Pault/DiffEngine.swift`:

```

//
// DiffEngine.swift
// Pault
//
// Two-pass diff: line-level first, then character-level within changed lines.
// Uses Swift's built-in CollectionDifference (Myers algorithm under the hood).
//

import Foundation

enum DiffEngine {

    enum DiffKind: Equatable {
        case unchanged
    }
}

```

```

case removed
case added
}

struct LineDiff: Identifiable {
  let id = UUID()
  let text: String
  let kind: DiffKind
  let characterDiffs: [CharacterDiff]?
}

struct CharacterDiff: Identifiable {
  let id = UUID()
  let text: String
  let kind: DiffKind
}

// MARK: - Line-level diff with character refinement

static func diff(old: String, new: String) -> [LineDiff] {
  let oldLines = old.isEmpty ? [] : old.components(separatedBy: "\n")
  let newLines = new.isEmpty ? [] : new.components(separatedBy: "\n")

  let changes = newLines.difference(from: oldLines).inferringMoves()

  // Build list of operations in order
  var removals: [Int: String] = [:] // offset -> element
  var insertions: [(offset: Int, element: String)] = []

  for change in changes {
    switch change {
    case .remove(let offset, let element, _):
      removals[offset] = element
    case .insert(let offset, let element, _):
      insertions.append((offset, element))
    }
  }

  // Walk through old lines, interleaving changes
  var result: [LineDiff] = []
  var insertionIndex = 0
  let sortedInsertions = insertions.sorted(by: { $0.offset < $1.offset })

  // Track which old-line offsets are removed
  let removedOffsets = Set(removals.keys)

  var newLineIdx = 0
  var oldLineIdx = 0

  // Use a unified approach: walk new lines, matching against old
  while oldLineIdx < oldLines.count || newLineIdx < newLines.count {
    if removedOffsets.contains(oldLineIdx) && newLineIdx < newLines.count {
      // Check if this is a modification (remove + insert at same position)
      let removedText = oldLines[oldLineIdx]
      let insertedText = newLines[newLineIdx]

      if removedText != insertedText {
        let charDiffsOld = characterDiff(old: removedText, new: insertedText)

```

```

let charDiffsNew = characterDiff(old: removedText, new: insertedText)
result.append(LineDiff(text: removedText, kind: .removed, characterDiffs: charDiffsOld))
result.append(LineDiff(text: insertedText, kind: .added, characterDiffs: charDiffsNew))
} else {
result.append(LineDiff(text: removedText, kind: .unchanged, characterDiffs: nil))
}
oldLineIdx += 1
newLineIdx += 1
} else if removedOffsets.contains(oldLineIdx) {
result.append(LineDiff(text: oldLines[oldLineIdx], kind: .removed, characterDiffs: nil))
oldLineIdx += 1
} else if oldLineIdx < oldLines.count && newLineIdx < newLines.count &&
oldLines[oldLineIdx] == newLines[newLineIdx] {
result.append(LineDiff(text: oldLines[oldLineIdx], kind: .unchanged, characterDiffs:
nil))
oldLineIdx += 1
newLineIdx += 1
} else if newLineIdx < newLines.count {
result.append(LineDiff(text: newLines[newLineIdx], kind: .added, characterDiffs: nil))
newLineIdx += 1
} else if oldLineIdx < oldLines.count {
result.append(LineDiff(text: oldLines[oldLineIdx], kind: .removed, characterDiffs: nil))
oldLineIdx += 1
}
}

return result
}

// MARK: - Character-level diff

static func characterDiff(old: String, new: String) -> [CharacterDiff] {
let oldChars = Array(old)
let newChars = Array(new)
let changes = newChars.difference(from: oldChars)

var result: [CharacterDiff] = []
var oIdx = 0

for change in changes {
switch change {
case .remove(let offset, let element, _):
while oIdx < offset {
result.append(CharacterDiff(text: String(oldChars[oIdx]), kind: .unchanged))
oIdx += 1
}
result.append(CharacterDiff(text: String(element), kind: .removed))
oIdx += 1
case .insert(_, let element, _):
result.append(CharacterDiff(text: String(element), kind: .added))
}
}
// Remaining unchanged characters
while oIdx < oldChars.count {
result.append(CharacterDiff(text: String(oldChars[oIdx]), kind: .unchanged))
oIdx += 1
}
}

```

```

return result
}
}

```

Note: The line-level diff uses a simplified walk-through approach. The character-level diff reuses the same `CollectionDifference` pattern from `RefinementLoopView.swift:18-44` but at character granularity. This implementation may need refinement during testing — the exact walk-through logic for matching removed/inserted line pairs is the trickiest part and should be validated carefully against the test cases.

Step 4: Run tests

Run: `xcodebuild test -scheme Pault -destination 'platform=macOS' -only-testing:PaultTests/DiffEngineTests 2>&1 | tail -30`

Expected: All tests PASS. If any fail, adjust the walk-through logic in `diff(old:new:)`.

Step 5: Commit

```

git add Pault/DiffEngine.swift PaultTests/DiffEngineTests.swift
git commit -m "feat(versioning): add DiffEngine with line+character-level diffing"

```

Task 4: Configurable Pruning Limit in Preferences

Files:

- Modify: `Pault/PreferencesView.swift:51-69` (`generalTab`)
- Modify: `Pault/PromptDetailView.swift:276` (pass limit to `saveSnapshot`)

Step 1: Add stepper to `generalTab`

In `Pault/PreferencesView.swift`, add `@AppStorage` at line 18 (after existing `@AppStorage` declarations):

```
@AppStorage("versionHistoryLimit") private var versionHistoryLimit: Int = 50
```

In the `generalTab` Form (around line 63, after the `defaultAction` Picker), add:

```
Stepper("Max versions per prompt: \(versionHistoryLimit)", value: $versionHistoryLimit,
in: 5...200)
```

Step 2: Wire limit to `saveSnapshot` call site

In `Pault/PromptDetailView.swift`, find the `saveSnapshot` call at line 276. Add `@AppStorage("versionHistoryLimit") private var versionHistoryLimit: Int = 50` to the view's properties, then update the call:

```
service.saveSnapshot(for: prompt, limit: versionHistoryLimit)
```

Step 3: Manual test

Build and run the app. Open Preferences > General. Verify the stepper appears and adjusts between 5-200.

Step 4: Commit

```

git add Pault/PreferencesView.swift Pault/PromptDetailView.swift
git commit -m "feat(versioning): add configurable version history limit in Preferences"

```

Task 5: Version Count Badge in Inspector

Files:

- Modify: `Pault/InspectorView.swift:30-34`

Step 1: Add version count to the History tab label

Replace the tab picker at `Pault/InspectorView.swift:30-34`:

```
Picker("", selection: $selectedTab) {
  ForEach(InspectorTab.allCases, id: \.self) { tab in
    if tab == .history {
      Text("\(tab.rawValue) (\(prompt.versions.count))").tag(tab)
    } else {
      Text(tab.rawValue).tag(tab)
    }
  }
}
```

Step 2: Manual test

Build and run. Create a prompt, edit it a few times. Check that the Inspector's History tab shows "History (3)" etc.

Step 3: Commit

```
git add Pault/InspectorView.swift
git commit -m "feat(versioning): show version count badge on History inspector tab"
```

Task 6: Version Management — Delete & Search

Files:

- Modify: `Pault/PromptVersionHistoryView.swift`

Step 1: Add delete and search functionality

Replace `Pault/PromptVersionHistoryView.swift` with the enhanced version that includes:

- `@State private var searchText: String = ""`
- Search bar at top of version list
- Swipe-to-delete on version rows
- Filtered versions computed property

```
//
// PromptVersionHistoryView.swift
// Pault
//

import SwiftUI
import SwiftData

struct PromptVersionHistoryView: View {
  @Bindable var prompt: Prompt
  @Environment(\.modelContext) private var modelContext
  @State private var selectedVersion: PromptVersion?
  @State private var searchText: String = ""
```

```
@State private var compareMode: Bool = false
@State private var compareSelections: Set<UUID> = []

private var versions: [PromptVersion] {
    var result = prompt.versions.sorted { $0.savedAt > $1.savedAt }
    if !searchText.isEmpty {
        let query = searchText.lowercased()
        result = result.filter {
            ($0.changeNote ?? "").lowercased().contains(query) ||
            $0.savedAt.formatted(date: .abbreviated, time: .shortened).lowercased().contains(query)
        }
    }
    return result
}

var body: some View {
    if prompt.versions.isEmpty {
        emptyState
    } else {
        VStack(spacing: 0) {
            // Search bar
            HStack(spacing: 6) {
                Image(systemName: "magnifyingglass")
                    .foregroundColor(.secondary)
                    .font(.caption)
                TextField("Search versions...", text: $searchText)
                    .textFieldStyle(.plain)
                    .font(.caption)
            }
            .padding(.horizontal, 12)
            .padding(.vertical, 6)
            .background(Color.secondary.opacity(0.08))

            // Compare mode toggle
            HStack {
                if compareMode {
                    Button("Cancel") {
                        compareMode = false
                        compareSelections.removeAll()
                    }
                    .font(.caption)
                }
                Spacer()
                Button("Compare (\(compareSelections.count)/2)") {
                    openComparison()
                }
                .font(.caption)
                .disabled(compareSelections.count != 2)
            } else {
                Spacer()
                Button {
                    compareMode = true
                } label: {
                    Image(systemName: "arrow.left.and.right")
                    .font(.caption)
                }
                .buttonStyle(.plain)
                .help("Compare two versions")
            }
        }
    }
}
```

```

}
.padding(.horizontal, 12)
.padding(.vertical, 4)

Divider()

// Version list
versionList
}
.sheet(item: $selectedVersion) { version in
PromptDiffView(version: version, prompt: prompt)
}
}
}

private var emptyState: some View {
VStack(spacing: 8) {
Image(systemName: "clock.arrow.circlepath")
.font(.title2)
.foregroundColor(.secondary)
Text("No history yet")
.font(.caption)
.foregroundColor(.secondary)
}
.frame(maxWidth: .infinity, maxHeight: .infinity)
.padding()
}

private var versionList: some View {
ScrollView {
LazyVStack(alignment: .leading, spacing: 0) {
ForEach(versions) { version in
HStack {
if compareMode {
Image(systemName: compareSelections.contains(version.id) ? "checkmark.circle.fill" :
"circle")
.foregroundColor(compareSelections.contains(version.id) ? .blue : .secondary)
.font(.caption)
}
VersionRow(version: version)
}
.contentShape(Rectangle())
.onTapGesture {
if compareMode {
toggleCompareSelection(version.id)
} else {
selectedVersion = version
}
}
.contextMenu {
Button("Delete", role: .destructive) {
deleteVersion(version)
}
}
Divider()
}
}
}
}

```

```

}

private func deleteVersion(_ version: PromptVersion) {
    modelContext.delete(version)
    try? modelContext.save()
}

private func toggleCompareSelection(_ id: UUID) {
    if compareSelections.contains(id) {
        compareSelections.remove(id)
    } else if compareSelections.count < 2 {
        compareSelections.insert(id)
    }
}

private func openComparison() {
    // Will be wired in Task 8 when PromptDiffView supports two-version comparison
    guard compareSelections.count == 2 else { return }
    let selected = versions.filter { compareSelections.contains($0.id) }
        .sorted { $0.savedAt < $1.savedAt }
    if let older = selected.first {
        selectedVersion = older // For now, open the older version in diff view
    }
}

// MARK: - VersionRow

private struct VersionRow: View {
    let version: PromptVersion

    var body: some View {
        HStack {
            VStack(alignment: .leading, spacing: 2) {
                Text(version.savedAt.formatted(date: .abbreviated, time: .shortened))
                    .font(.caption)
                    .foregroundColor(.primary)
                if let note = version.changeNote, !note.isEmpty {
                    Text(note)
                        .font(.caption2)
                        .foregroundColor(.secondary)
                        .lineLimit(1)
                }
            }
            Spacer()
            Image(systemName: "chevron.right")
                .font(.caption2)
                .foregroundColor(.tertiary)
        }
        .padding(.horizontal, 12)
        .padding(.vertical, 8)
    }
}

```

Step 2: Manual test

Build and run. Create a prompt, make several edits. Open version history.

- Verify search bar filters by change note text.
- Right-click a version > Delete — verify it's removed.
- Tap "compare" icon — verify checkboxes appear, can select two versions.

Step 3: Commit

```
git add Pault/PromptVersionHistoryView.swift
git commit -m "feat(versioning): add version search, context-menu delete, and compare mode"
```

Task 7: Enhanced PromptDiffView with Inline/Side-by-Side Toggle

Files:

- Modify: Pault/PromptDiffView.swift

Step 1: Rewrite PromptDiffView with DiffEngine integration

This is the largest UI change. Replace Pault/PromptDiffView.swift entirely.

The new implementation should include:

- @State private var diffMode: DiffMode = .sideBySide with enum .inline, .sideBySide
- Segmented control toggle at the top
- Inline mode: single scrollable pane using DiffEngine.diff(old:new:), showing removed lines with .red.opacity(0.15) background, added lines with .green.opacity(0.15) background, character-level diffs as bold text within each line
- Side-by-side mode: two-pane layout (current structure) but enhanced with line-level highlighting using DiffEngine
- Metadata changes section (collapsible): shows tag adds/removes, variable changes, favorite status change — comparing version.snapshot vs current prompt
- Restore button opens a confirmation sheet (Task 8) instead of immediate restore

Key implementation detail — the inline diff rendering:

```
// For each LineDiff, render with appropriate background and character highlights
ForEach(diffs) { lineDiff in
  HStack(spacing: 0) {
    if let charDiffs = lineDiff.characterDiffs {
      charDiffs.reduce(Text("")) { partial, cd in
        switch cd.kind {
          case .unchanged: partial + Text(cd.text)
          case .removed: partial + Text(cd.text).fontWeight(.bold).foregroundColor(.red)
          case .added: partial + Text(cd.text).fontWeight(.bold).foregroundColor(.green)
        }
      }
    } else {
      Text(lineDiff.text)
    }
  }
  .frame(maxWidth: .infinity, alignment: .leading)
  .padding(.vertical, 2)
  .padding(.horizontal, 8)
  .background(lineBackground(for: lineDiff.kind))
}
```

The full implementation is ~200 lines. Write it to include:

1. DiffMode enum and segmented control
2. diffs computed property calling `DiffEngine.diff(old: version.content, new: prompt.content)`
3. Inline view using the pattern above
4. Side-by-side view with highlighting
5. Metadata changes section comparing `version.snapshot` to current prompt state
6. Restore button that sets `@State private var showRestorePreview = true`

Step 2: Manual test

Build and run. Open a version from history.

- Toggle between Inline and Side-by-Side modes
- Verify green/red highlighting appears for changed lines
- Verify character-level bold highlights within modified lines
- If version has metadata snapshot, verify metadata changes section shows tag/variable diffs

Step 3: Commit

```
git add Pault/PromptDiffView.swift
git commit -m "feat(versioning): overhaul diff view with inline/side-by-side toggle and DiffEngine"
```

Task 8: Restore Preview with Partial Restore

Files:

- Modify: `Pault/PromptDiffView.swift` (add restore preview sheet)

Step 1: Add restore preview sheet

Add to `PromptDiffView`:

- `@State private var showRestorePreview = false`
- `@State private var restoreContent = true`
- `@State private var restoreTitle = true`
- `@State private var restoreTags = true`
- `@State private var restoreVariables = true`
- `@State private var restoreFavorite = true`

The "Restore This Version" button now sets `showRestorePreview = true` instead of calling `restoreVersion()` directly.

The restore preview sheet shows:

1. A diff summary (using `DiffEngine`) of what will change
2. Checkboxes for each field: Content, Title, Tags, Variables, Favorite
3. Each checkbox is disabled if that field hasn't changed
4. "Confirm Restore" button applies only the selected fields

Restore logic (partial):

```

private func performRestore() {
// Snapshot current state before restore
service.saveSnapshot(for: prompt, changeNote: "Before restore from \$(dateString)")

if restoreTitle { prompt.title = version.title }
if restoreContent {
prompt.content = version.content
prompt.attributedContent = nil
}
if restoreFavorite { prompt.isFavorite = version.isFavorite }
if restoreTags, let snap = version.snapshot {
// Remove current tags
prompt.tags.removeAll()
// Re-add from snapshot
for tagSnap in snap.tags {
let tag = service.createTag(name: tagSnap.name, color: tagSnap.color)
service.addTag(tag, to: prompt)
}
}
if restoreVariables, let snap = version.snapshot {
// Sync variables from content first (if content was also restored)
if restoreContent {
TemplateEngine.syncVariables(for: prompt, in: modelContext)
}
// Overlay snapshot defaults
for varSnap in snap.variables {
if let existing = prompt.templateVariables.first(where: {
$0.name == varSnap.name && $0.occurrenceIndex == varSnap.occurrenceIndex
}) {
existing.defaultValue = varSnap.defaultValue
}
}
}

prompt.updatedAt = Date()
service.saveSnapshot(for: prompt, changeNote: "Restored from \$(dateString)")
dismiss()
}

```

Step 2: Manual test

Build and run. Open a version, click "Restore This Version".

- Verify preview sheet appears showing what will change
- Uncheck "Title" — restore only content and other fields
- Click Confirm — verify only selected fields are restored
- Check version history — verify "Before restore" and "Restored from" snapshots both appear

Step 3: Commit

```

git add Pault/PromptDiffView.swift
git commit -m "feat(versioning): add restore preview sheet with partial field selection"

```

Task 9: Run Full Test Suite + Final Verification

Files: None (verification only)

Step 1: Run all tests

Run: `xcodebuild test -scheme Pault -destination 'platform=macOS' 2>&1 | tail -40`

Expected: All tests PASS.

Step 2: Full manual smoke test

Follow the verification checklist from the design doc:

1. Create a prompt, edit it several times → versions appear with metadata
2. Open diff view → line/character highlighting in both modes
3. Compare two arbitrary versions → correct diff
4. Delete versions via context menu → removed
5. Search versions by change note → filtered
6. Restore with preview → diff shown, partial fields work
7. Restore tags that no longer exist → recreated
8. Set pruning limit in Preferences → older versions pruned
9. Rapid typing → dedup guard prevents duplicate snapshots
10. Version count badge updates in Inspector

Step 3: Commit any fixes

If any issues found during verification, fix and commit with descriptive messages.

Step 4: Final commit

```
git add -A
git commit -m "chore(versioning): finalize versioning v2 overhaul"
```